

HLD Assignment

End-Term

Search Typeahead System

A search-as-you-type backend with a distributed cache, consistent hashing, recency-aware trending, and batched writes.

Project Report

Date: 22 June 2026

Stack: Node.js 18+ · Express · in-memory data structures

Contents

1. System Overview & Architecture
2. Dataset — Source & Loading Instructions
3. API Documentation
4. Design Choices & Trade-offs
5. Performance Report

1. System Overview & Architecture

A search-as-you-type system: as the user types a prefix, the server returns the 10 most relevant matching queries. Submitting a search records it, which feeds back into popularity and trending rankings.

The core of the system is its backend data design, built around four ideas:

- How query-count data is **stored** and indexed for fast prefix lookup.
- A **distributed cache** (multiple logical nodes) fronted by **consistent hashing**.
- A **recency-aware trending** ranking layered on top of all-time popularity.
- **Batch writes** so the store is not hit on every single search.

Architecture diagram



Components

Component	File	Responsibility
Primary store	src/store.js	Source of truth for counts; owns the prefix index.
Trie	src/trie.js	"starts-with" (prefix) lookup.
Consistent hash ring	src/cache/consistentHash.js	Maps a cache key → owning cache node.
Distributed cache	src/cache/distributedCache.js	N cache nodes, each with TTL + LRU eviction.
Batch writer	src/batchWriter.js	Buffers and aggregates writes; invalidates cache.
HTTP server	src/server.js	Routes and wiring.

Config	<code>src/config.js</code>	Tunable knobs (node count, TTL, batch size, half-life).
--------	----------------------------	---

Request flows

Read path (GET /suggest): build the key `rank::prefix` → consistent hashing selects the cache node → look up. On a *hit*, return immediately. On a *miss*, walk the trie for candidates, read their counts from the Map, rank, write the result back into the cache, and return.

Write path (POST /search): the request only buffers the submission in the batch writer and returns immediately. A flush (triggered by buffer size or a 2-second timer) applies aggregated counts to the store once and invalidates the affected prefix cache keys.

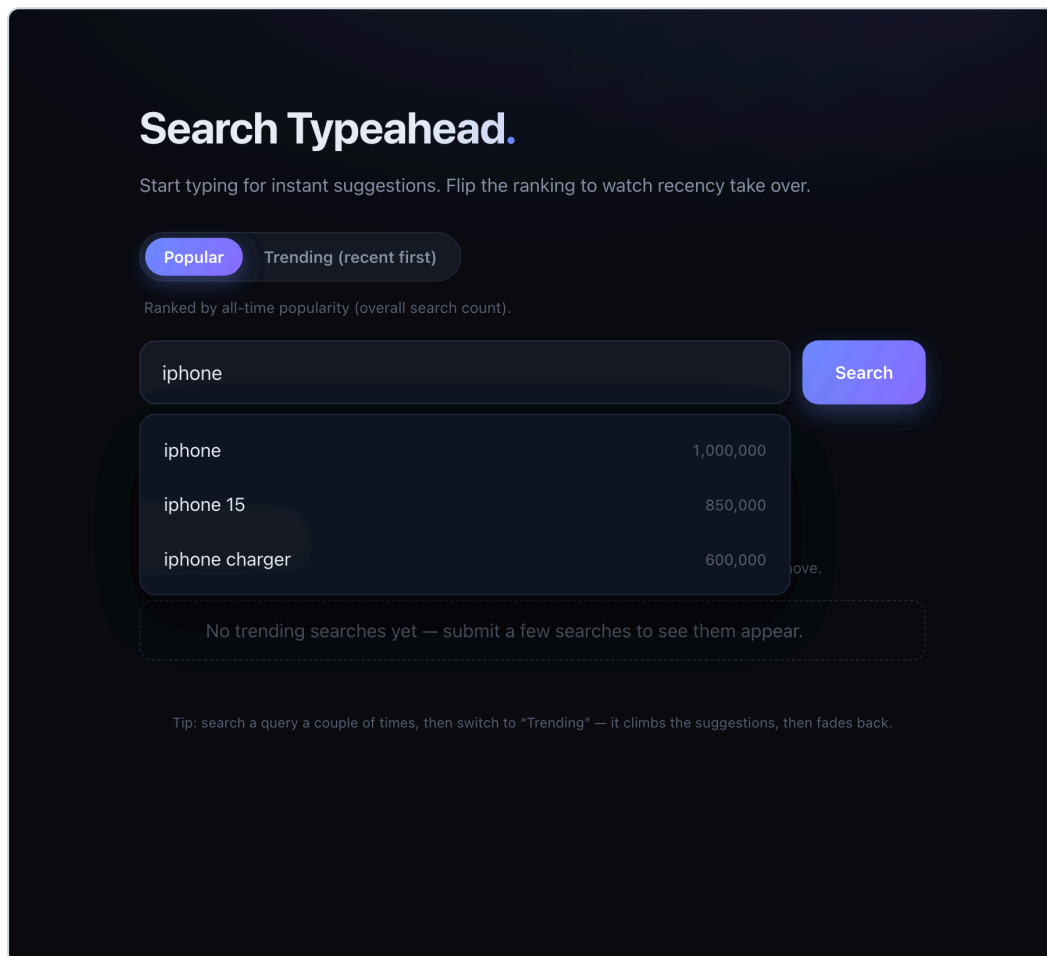


Figure 1 — The running app: live suggestions for the prefix "iphone", ranked by popularity.

2. Dataset — Source & Loading Instructions

Source

The dataset is a **synthetic, reproducible e-commerce / tech query corpus** generated by `scripts/generate-dataset.js`. The assignment permits any dataset with a `query, count` shape and a minimum of 100,000 rows; this generator produces **120,000 unique queries** (comfortably above the minimum).

- **Construction:** queries are combinations of 39 brands, 80 product categories, and 63 modifiers (e.g. `apple laptop pro`, `nike running shoes wireless`), plus a set of hand-picked "head" queries (`iphone` = 1,000,000, `airpods` = 720,000, ...).
- **Count distribution:** a skewed, Zipf-like distribution so short, generic queries dominate and a long tail trails off — mirroring real search traffic and making the typeahead demo realistic.
- **Reproducibility:** a seeded `mulberry32` RNG (seed = 42) makes the generated file identical across machines and runs.

Using a real dataset instead: drop any CSV with a `query, count` header at `data/queries.csv` (e.g. AOL query logs, Wikipedia page titles, Amazon product titles) and skip the generate step.

Format

```
query, count
iphone, 1000000
iphone 15, 850000
iphone charger, 600000
java tutorial, 400000
...
```

Queries may contain spaces but never commas, so plain CSV parsing is safe.

Loading instructions

```
npm install      # install express
npm run generate-data # build data/queries.csv (120,000 rows)
npm start        # start server on http://localhost:3000
```

On startup the server streams `data/queries.csv` into the primary store: each row populates the `Map<query, {count, ...}>` and inserts the query into the trie, building the prefix index in memory.

3. API Documentation

Base URL: `http://localhost:3000`. All responses are JSON. All edge cases (missing/empty params, no matches, mixed case) are handled gracefully — no 500s.

GET /suggest

Return up to 10 suggestions whose query starts with the given prefix.

Param	Required	Default	Description
q	yes	—	The typed prefix (case-insensitive).
rank	no	popular	popular (all-time count) or recent (recency-aware).

```
curl "http://localhost:3000/suggest?q=iph&rank=popular"

{
  "prefix": "iph", "rank": "popular", "cached": false,
  "suggestions": [
    { "query": "iphone", "count": 1000000, "recent": 0, "score": 1000000 },
    { "query": "iphone 15", "count": 850000, "recent": 0, "score": 850000 },
    { "query": "iphone charger", "count": 600000, "recent": 0, "score": 600000 }
  ]
}
```

`cached` indicates a cache vs store hit; `score` is the ranking value (equals `count` in popular mode); `recent` is the decayed recent-activity score.

POST /search

Record a submitted search (enters the batch writer; affects rankings after the next flush).

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" -d '{"query":"iphone 15"}'

{ "message": "Searched", "query": "iphone 15" }
```

New queries are inserted; existing counts incremented. Empty/missing query → `400 { "error": "query is required" }`.

GET /trending

Currently-hot queries ranked purely by decayed recent activity. Param `limit` (default 10, max 50).

```
curl "http://localhost:3000/trending?limit=5"

{ "trending": [
  { "query": "iphone charger", "recentScore": 7.68, "count": 600008 },
  { "query": "nike shoes", "recentScore": 2.49, "count": 300003 }
] }
```

GET /cache/debug

Shows how a prefix key is routed by consistent hashing and whether it is cached. Params: `prefix` (required), `rank` (default `popular`).

```
curl "http://localhost:3000/cache/debug?prefix=iph"

{ "key": "popular::iph", "keyHash": 3628440234,
  "node": "cache-node-2", "status": "hit", "expiresInMs": 29866 }
```

GET /stats

Reports cache hit rate + per-node spread, store read/write counters, and batch write-reduction.

```
{
  "store": { "totalQueries": 120000, "dbReads": 62, "dbWrites": 7 },
  "cache": { "hits": 1438, "misses": 60, "hitRate": 0.96, "totalKeys": 44,
    "perNode": [ { "node": "cache-node-0", "keys": 12, "hitRate": 0.957 } ] },
  "batch": { "submissionsReceived": 1008, "flushCount": 2, "rowsWritten": 7,
    "writeReductionFactor": 144 }
}
```

POST /admin/flush

Force an immediate batch flush (useful in demos so rankings update without waiting for the timer).

```
curl -X POST http://localhost:3000/admin/flush
{ "flushed": 3, "reason": "manual" }
```

4. Design Choices & Trade-offs

4.1 Storage & prefix index — Map + Trie

Two structures sit side by side: a `Map<query, {count, recentScore, lastDecayAt}>` as the source of truth ($O(1)$ exact lookups/updates), and a **trie** as the prefix index. Typeahead is fundamentally a prefix problem; a trie walks straight to a prefix node and collects the queries beneath it, and new queries insert character-by-character without re-sorting.

Why counts live in the Map, not the trie: separation of concerns — the trie answers *membership*, the Map answers *popularity*. A count update is then a single Map write, not a tree mutation. A `maxScan` safety cap bounds how many candidates one lookup gathers so a pathologically broad prefix can't walk the whole tree.

Alternative considered: sorted array + binary search for the prefix range — lower memory, but inserting a brand-new query is $O(n)$. The trie wins because `/search` can introduce new queries.

4.2 Consistent hashing

Problem: naive `hash(key) % nodeCount` remaps almost every key when the node count changes, flushing the whole cache. **Solution:** place each node at many points on a circular keyspace; to find an owner, hash the key and walk clockwise to the first node point. Adding/removing a node only moves keys in one arc.

- MD5 hash, first 32 bits as an unsigned int — even spread, no crypto strength needed.
- Ring is a sorted array; lookup is a **binary search** ($O(\log R)$) with wrap-around.
- **150 virtual nodes** per physical node smooth the distribution; with only 4 nodes and no replicas the arcs would be very uneven.

4.3 Distributed cache (TTL + LRU)

Caches the *result* of a lookup, keyed by `rank::prefix`, spread across 4 logical nodes. Each node is an in-memory Map with a **30s TTL** (checked lazily on read) and an **LRU bound** (insertion-ordered Map; oldest key evicted on overflow; reads re-insert to mark most-recently-used). The rank mode is part of the key because `popular` and `recent` produce different lists.

Invalidation: on a batch flush, for each changed query the cache entries for its prefixes (lengths 1–15, both rank modes) are deleted — *targeted* invalidation rather than wiping the whole cache; everything else expires via TTL. **Trade-off:** a few `delete` s per changed query, cheap because flushes are batched and aggregated.

4.4 Trending (recency-aware ranking)

Enhanced score: `log10(count + 1) + recencyWeight × decayedRecentScore`, where `decayedRecentScore = recentScore × 0.5^(elapsed / halfLife)`.

- **Why log the count?** Counts span 1 to 1,000,000; logging compresses popularity to ~0–6 so a burst of recent searches (weight 3 each) can actually reorder results, even lifting a freshly-searched query above all-time leaders.
- **No permanent over-ranking:** the recency term decays exponentially (5-minute half-life), so a once-hot query drifts back to its historical rank on its own — nothing has to actively demote it.

- **Lazy decay:** scores are recomputed on read/update (one `Math.pow`), so there is no background scheduler.

Trade-offs: shorter TTL / aggressive invalidation = fresher rankings but more misses; lazy decay = simpler but recomputes on read; log-dampening = simple and explainable vs a more precise but heavier sliding-window model.

4.5 Batch writes

`/search` calls `record(query)`, which increments an in-memory `Map<query, pendingCount>` and returns immediately. **Aggregation** turns 200 searches for `iphone` into one `+200` update. Flush triggers: buffer reaches 50 distinct queries *or* a 2-second timer fires. On flush each distinct query is applied once and its prefix cache keys invalidated.

Failure trade-off: the buffer is in memory, so a crash between flushes loses at most ~2 seconds of count increments. Acceptable because counts are approximate popularity signals, not transactions, and self-correct as searches continue. A durable version would append each submission to a write-ahead log before buffering and replay on restart — trading write latency for durability.

4.6 Default configuration

Knob	Value	Reasoning
<code>cache.nodeCount</code>	4	Enough to show distribution without noise.
<code>cache.virtualNodes</code>	150	Smooth key spread across only 4 nodes.
<code>cache.ttlMs</code>	30s	Balance freshness vs hit rate.
<code>batch.maxSize</code>	50	Flush promptly under load.
<code>batch.flushIntervalMs</code>	2s	Bound staleness when traffic is light.
<code>trending.halfLifeMs</code>	5 min	A spike stays relevant for minutes, not forever.
<code>trending.recencyWeight</code>	3	A couple of recent searches can top all-time leaders.

5. Performance Report

Numbers produced by `npm run benchmark` against a locally running server (Node 26, Apple Silicon, 120,000-query dataset). Absolute numbers vary by machine, but the shape — cache hits an order of magnitude faster than misses, very high hit rate, large write reduction — holds.

~0.29 ms

/SUGGEST P95 (CACHE HIT)

~96%

CACHE HIT RATE

~144x

BATCH WRITE REDUCTION

5.1 Suggestion latency (/suggest)

Traffic	p50	p95	p99	max
Cache miss (cold prefixes)	0.374 ms	10.107 ms	42.398 ms	42.398 ms
Cache hit (warm prefixes)	0.136 ms	0.288 ms	0.527 ms	0.912 ms

The first request for a prefix pays for the trie walk + ranking; once cached, repeats are served in well under a millisecond — a **~35x p95 improvement**. Short prefixes (huge candidate sets) dominate the cache-miss tail, exactly the cost the cache is there to hide.

5.2 Cache hit rate & key distribution

```
hit rate: 96.0%
total keys cached: 44
per-node key spread:
  cache-node-0: 12 keys, 95.7% hit rate
  cache-node-1: 12 keys, 95.6% hit rate
  cache-node-2:  8 keys, 97.3% hit rate
  cache-node-3: 12 keys, 95.4% hit rate
```

A **96% hit rate** on typeahead traffic (the same prefixes are typed repeatedly), so the store is read only on the ~4% cold/expired requests. Consistent hashing with 150 virtual nodes spreads keys roughly evenly (~12/12/8/12) with no hot-spotting.

5.3 Store read/write counters

```
total queries: 120,000
db reads (suggest fell through to store): 62
db writes (rows touched): 7
```

Of ~1,500 suggestion requests in the run, only **62 reached the primary store** — the cache absorbed the rest.

5.4 Batch write reduction

```
search submissions: 1008
rows actually written: 7
flushes: 2
write reduction factor: 144x
```

1,008 `/search` submissions (with heavy repeats, as real traffic has) collapsed into just **7 store writes** via aggregation — a **~144x** reduction.

Summary

Metric	Result
/suggest p95 — cache hit	~0.29 ms
/suggest p95 — cache miss	~10 ms
Cache hit rate	~96%
Key spread (4 nodes)	~12 / 12 / 8 / 12
Store reads avoided by cache	~96% of requests
Batch write reduction	~144x

Reproduce

```
npm install
npm run generate-data
npm start          # terminal 1
npm run benchmark  # terminal 2
```